

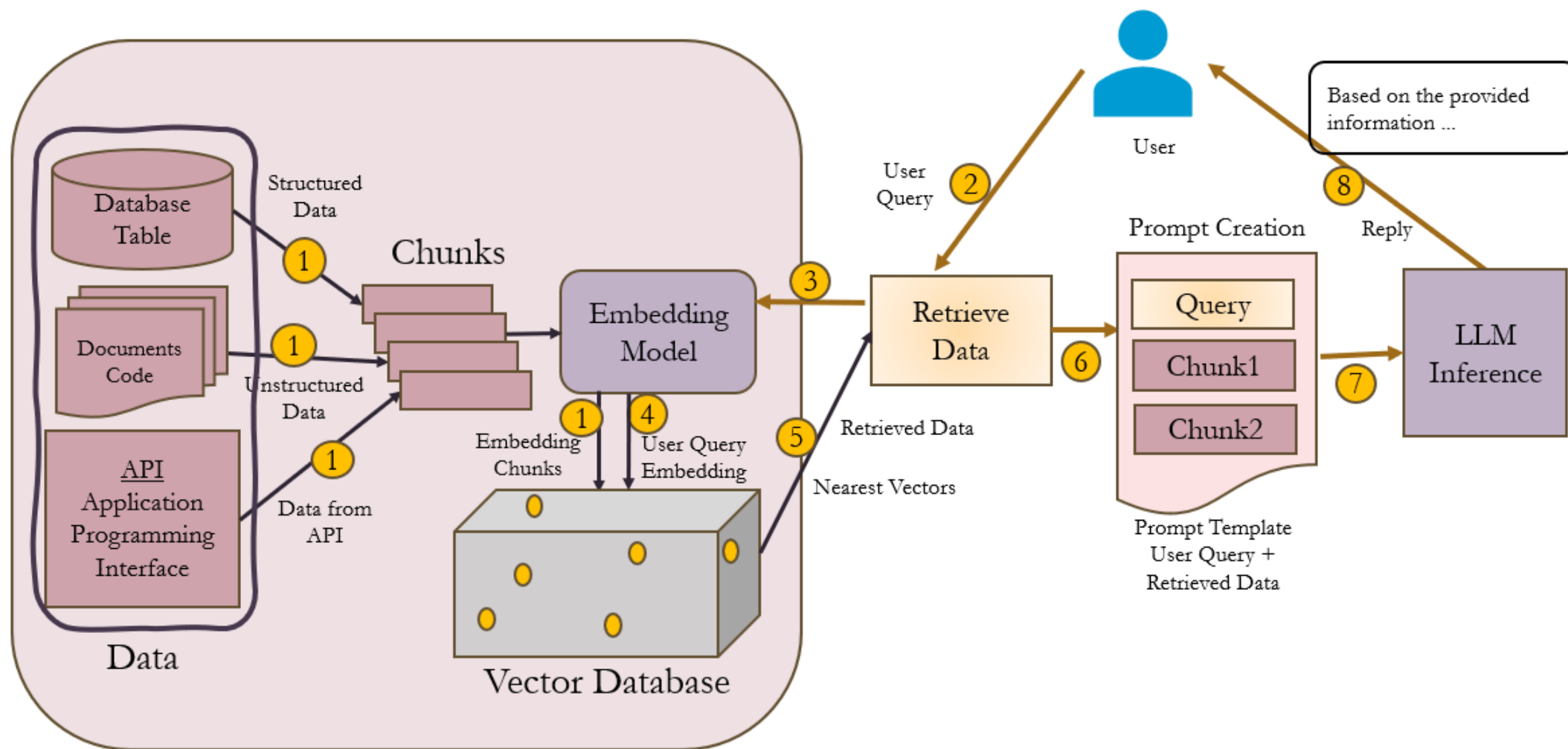
GenAI LLM

RAG & Multimodal

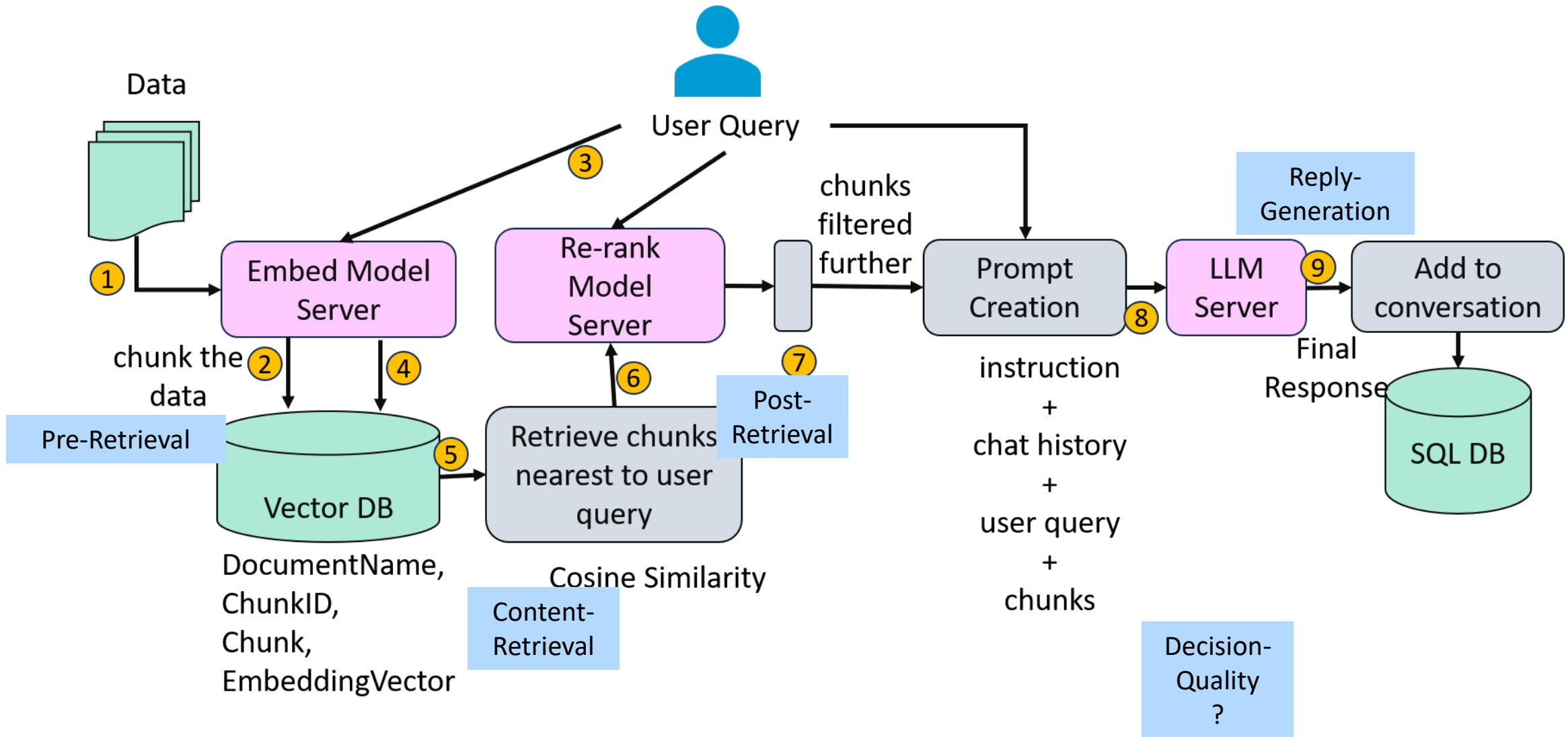
Session 4

Retrieval Augmented Generation

RAG



Enhanced RAG



Indexing

Inverted File Index (IVF)

IVF accelerates search by partitioning the search space into smaller, manageable regions called clusters (or Voronoi cells).

How it works:

- During the indexing phase, the algorithm uses clustering (like k-means) to group similar vectors together and calculates a "centroid" for each cluster.
- When a search query is initiated, the system first compares the query vector to the centroids.
- It then limits the actual search to only the vectors within the nearest cluster(s).

Advantage

- Drastically reduces the number of distance calculations needed, speeding up search times compared to an exhaustive search.

Hierarchical Navigable Small World (HNSW)

HNSW is a graph-based indexing algorithm that is widely considered state-of-the-art for vector search due to its excellent balance of speed and accuracy.

How it works:

- It builds a multi-layered graph hierarchy.
- The top layers contain very few, highly disconnected nodes, while the bottom layers contain dense, highly connected nodes.
- A search starts at the top layer to quickly navigate across the dataset over long distances.
- As it gets closer to the target, it drops down to lower layers for fine-grained, localized searching among neighbors.

Advantage:

- Exceptionally fast search speeds with very high recall (accuracy).

Product Quantization (PQ)

PQ is primarily a compression technique used to reduce the memory footprint of high-dimensional vectors.

How it works:

- It splits a large, high-dimensional vector into smaller, equal-sized sub-vectors.
- It then clusters these sub-vectors and replaces them with a short identifier (the ID of the nearest cluster centroid).
- During a search, distances are approximated using these compressed representations rather than the raw data.

Advantage:

- Massively reduces RAM requirements and storage costs.
- It is often combined with IVF (as IVF-PQ) to achieve both fast search and low memory usage.

Locality-Sensitive Hashing (LSH)

LSH is an algorithmic technique that groups similar items into the same "buckets" using specialized hash functions.

How it works:

- Unlike traditional cryptographic hashing (where slight input changes result in entirely different hashes), LSH is designed so that similar input vectors have a high probability of producing the same hash value.
- When a query comes in, it is hashed, and the system only compares the query to the vectors stored in the same (or neighboring) hash buckets.

Advantage:

- Highly scalable for massive datasets and effectively handles the "curse of dimensionality," though it can be complex to tune the hash functions for optimal accuracy.

Comparison

Feature	IVF (Inverted File Index)	HNSW (Hierarchical Graph)	PQ (Product Quantization)	LSH (Locality-Sensitive Hashing)
Primary Mechanism	Clustering (Voronoi partitions)	Multi-layered graphs	Vector compression/chunking	Hash functions & buckets
Search Speed	Fast	Extremely Fast	Fast (due to reduced data size)	Fast
Memory Usage	Moderate	High (needs to store graph links)	Very Low	Low to Moderate
Accuracy (Recall)	Moderate to High (depends on clusters searched)	Very High	Moderate (lossy compression limits precision)	Moderate (probabilistic)
Best Use Case	Datasets where a good balance of speed and memory is needed.	Scenarios demanding the highest accuracy and speed, where memory is not a constraint.	Massive datasets where fitting indexes into RAM is the primary bottleneck.	High-dimensional data where approximate probabilistic matching is sufficient.

Relevance Scoring & Ranking

Relevance Scoring Methods

Relevance Scoring

- Algorithms used in information retrieval, search engines, and database querying to calculate how well a specific document matches a user's search query.
- They assign a numerical score to each document so the system can rank and return the most relevant results first.

Methods

- TF-IDF (Term Frequency-Inverse Document Frequency)
- BM25 (Best Matching 25)
- Learning to Rank (LTR) algorithms

Relevance Scoring Methods

TF-IDF (Term Frequency-Inverse Document Frequency)

- How important a specific word is to a document within a larger dataset (or corpus)
- Determines relevance by multiplying two metrics:
 - Term Frequency (how often the word appears in the specific document)
 - Inverse Document Frequency (decrease weight of commonly used words, increase weight of rare words in dataset)

BM25 (Best Matching 25)

- Probabilistic ranking function, better than TF-IDF
- Two enhancements
 - Term frequency saturation (caps score so that a document doesn't get high score just because a keyword appears hundreds of times)
 - Document length normalization (adjusts scores so that exceptionally long documents don't have an unfair advantage over shorter ones)

Relevance Scoring Methods

Learning to Rank (LTR) algorithms

- Class of machine learning techniques used to solve ranking problems in information retrieval systems.
- Uses supervised machine learning models trained on historical data, such as user click-through rates or human-graded relevance labels.
- The model learns to simultaneously weigh dozens or hundreds of different features (including BM25 scores, text matches, document freshness, and user location) to predict the optimal order of search results.

Cross-encoder models & Pairwise scoring

Cross-encoder models:

- A type of neural network architecture where both the search query and the document are passed into the model simultaneously.
- This allows the model to deeply analyze the intricate relationship and context between the specific words in the query and the document, resulting in a highly accurate relevance score.

Pairwise relevance scoring:

- A ranking method where the model compares documents two at a time against the query.
- It evaluates whether Document A is more relevant than Document B for a given search, rather than just scoring each document in total isolation.

Example

- Used generally in use cases where vector database and LLM are required.

Contextual re-ranking & Language models

Contextual re-ranking:

- Re-ordering search results by evaluating the deeper semantic context, intent, and meaning of the text, rather than relying strictly on exact keyword overlaps.

Pre-trained language models:

- Large AI models (like BERT) that have been previously trained on vast amounts of text data to understand language structure, grammar, and semantics.
- These models are adapted to evaluate how well a document answers a specific query.

Example

- Help desks and automated support bots must handle highly technical, context-heavy queries where standard keyword matching fails
- e.g., differentiating between a guide on *how a feature works* versus a guide on *how to fix a bug in that feature*.

Diversity-aware re-ranking & Coverage

Diversity-aware re-ranking:

- A technique that deliberately adjusts the final ranking order to ensure the top results cover a variety of different sub-topics, perspectives, or interpretations of a query.
- This prevents a scenario where the top 10 results are essentially identical to one another.

Result set coverage:

- The extent to which the search results address all possible facets, meanings, or intents of a user's query, providing a comprehensive set of answers rather than a narrow one.

Example

- When a user types a broad, ambiguous query like "running shoes" into a storefront, presenting them with ten nearly identical black Nike shoes results in a poor user experience and lower conversion rates.

Re-ranking Techniques

Re-ranking Technique	Primary Domain	Real-World Implementation
Cross-Encoder Pairwise Scoring	Retrieval-Augmented Generation (RAG)	Serves as the critical bridge between a vector database and an LLM (using models like Cohere Rerank or BGE). It re-scores the top 50 retrieved documents to ensure the AI only generates answers based on the most accurate context.
Contextual Pre-Trained Models	Enterprise Knowledge Management	Deployed in platforms like Azure AI Search or Elasticsearch to parse complex, domain-specific language in company wikis, legal documents, or support tickets by understanding semantic intent rather than just keywords.
Diversity-Aware Re-ranking	E-commerce & Streaming RecSys	Utilized by recommendation algorithms (often using Maximal Marginal Relevance) to ensure a user searching for a broad term sees a varied mix of brands, genres, or price points, maximizing the chance of engagement.

Google Enterprise RAG

- Gemini models are natively multimodal.
 - Processes text, images, audio, and video directly within a unified latent space.
- Aggressively restricts the LLM to only use provided context, heavily penalizing hallucination.
- ScaNN (Scalable Nearest Neighbors)
 - Highly optimized vector search algorithm
 - Built on Inverted File with Product Quantization (IVFPQ)
- Relies heavily on Hybrid Search,
 - executing both dense semantic vector searches (via ScaNN) and sparse keyword searches concurrently.
 - combined results are passed through a cross-encoder model—a neural re-ranker—that scores and fuses the data to ensure the most relevant documents bubble to the top.
- Tensor Processing Units (TPUs)
 - Hardware Optimization
 - ensure that the massive compute required for real-time multimodal generation remains economically viable at scale

Advanced RAG Techniques

Query Rewriting

Query rewriting uses a Large Language Model (LLM) to transform the raw user input into a highly optimized, standalone search query before retrieving documents.

- often input queries that are conversational, ambiguous, or missing context
- "What does it do?" after a previous question

Steps

- Receive the raw user query (and conversational history, if applicable).
- Pass the query to an LLM prompted to act as a query optimizer. The LLM resolves pronouns, corrects typos, and adds missing context.
- Execute the search against the vector database using this newly generated, explicit query.
- Pass the retrieved documents and the query to the main LLM to generate the final response.

Multi-query RAG

Multi-query RAG (often called query expansion) generates multiple variations of the original query to cast a wider net

- single query might miss relevant documents because human language is nuanced; the exact wording matters in vector search

Steps

- Receive the original user query.
- Prompt an LLM to generate 3 to 5 differently worded variations of the query from different perspectives.
- Run parallel retrieval searches for *each* generated query variation.
- Aggregate and deduplicate the retrieved documents from all searches. Often, a ranking algorithm like Reciprocal Rank Fusion (RRF) is used to prioritize the most relevant results.
- Pass the consolidated, highly relevant context to the LLM to generate the answer.

Multi-hop RAG

Multi-hop RAG solves this by iteratively breaking the question down and performing sequential retrievals, where each "hop" depends on the information found in the previous one.

- Complex questions that require synthesizing information from entirely separate documents.
- "Who won the championship in the year the CEO of Company X was born?")

Steps

- Receive a complex, multi-part query.
- The LLM identifies the first piece of missing information needed to start answering the question and generates a sub-query for it.
- Retrieve context for the first sub-query.
- The LLM analyzes the retrieved information. If the overall question cannot yet be answered, it formulates the *next* logical sub-query based on the new context it just learned.
- Repeat steps 3 and 4 until all necessary facts are gathered.
- Synthesize the accumulated context into a final, comprehensive answer.

Query Routing

Query routing acts as a traffic controller, intelligently directing the user's prompt to the specific datastore or retrieval pipeline best suited to answer it.

- In enterprise environments, data lives in multiple places (vector databases for PDFs, SQL databases for structured metrics, graph databases for relationships)

Steps

- Receive the user query.
- Pass the query to a router (this can be a lightweight LLM, a classifier, or a set of semantic routing rules).
- The router analyzes the query's intent and selects the optimal destination (e.g., routing "summarize the employee handbook" to the vector DB, but routing "what were our Q3 sales?" to the SQL DB).
- Execute the retrieval using the specific rules and tools of the chosen datastore.
- Format the retrieved data and return it to the LLM for generation.

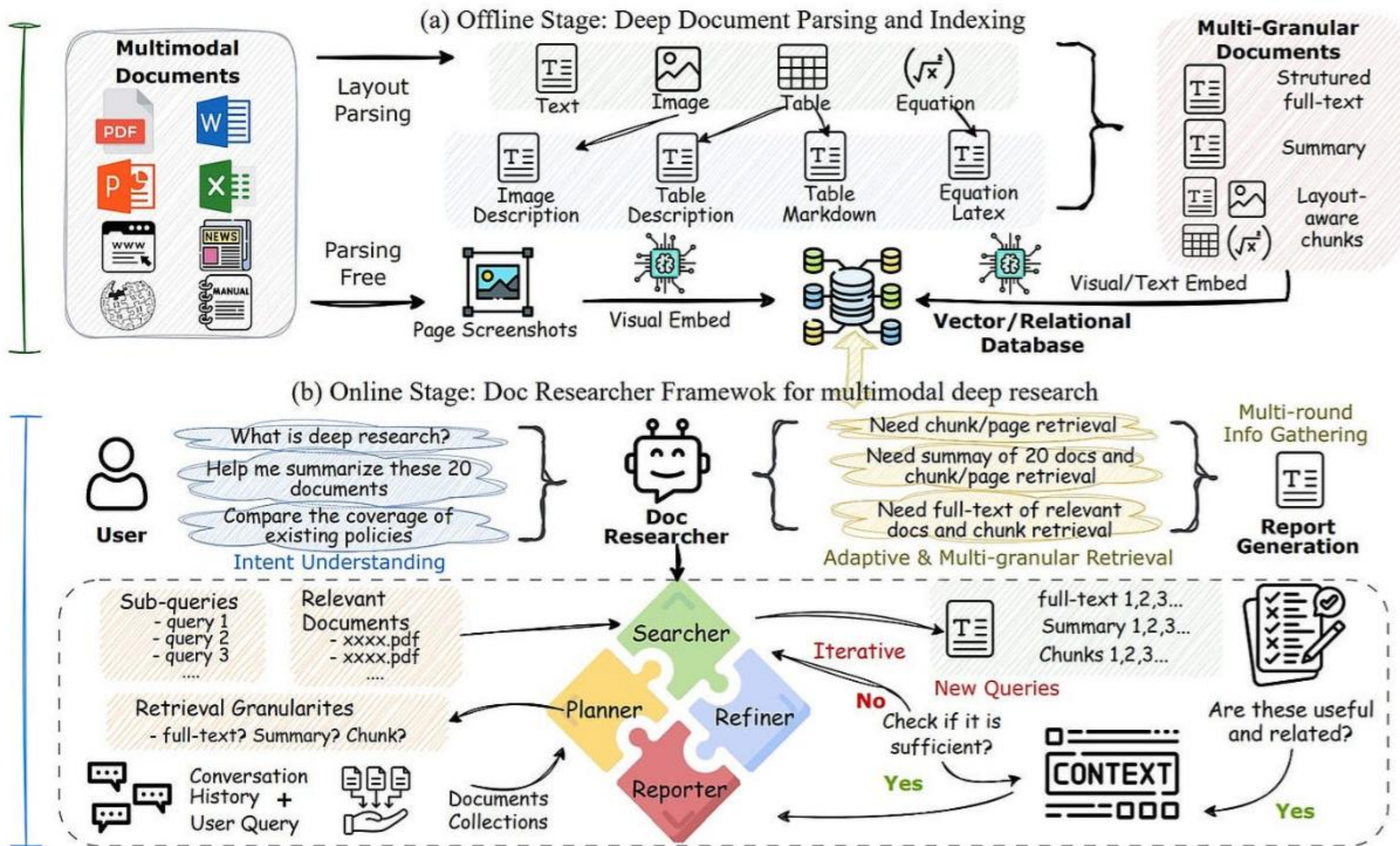
Agentic RAG

Unlike traditional RAG, which follows a rigid, linear pipeline (Retrieve -> Generate), Agentic RAG turns the LLM into an autonomous agent equipped with retrieval tools.

The agent actively decides *if*, *when*, and *how* to retrieve information, rather than being forced to retrieve data on every single turn.

Steps

- Receive the user query.
- The agent evaluates the query and determines if external knowledge is required. (If it already knows the answer or is just making casual conversation, it skips retrieval).
- If external data is needed, the agent selects the appropriate retrieval tool and crafts the query for it.
- The agent reviews the retrieved context. It then autonomously decides: "Do I have enough information to answer the user, or do I need to use the retrieval tool again to find more details?"
- Once the agent determines it has gathered sufficient context, it drafts and delivers the final response.

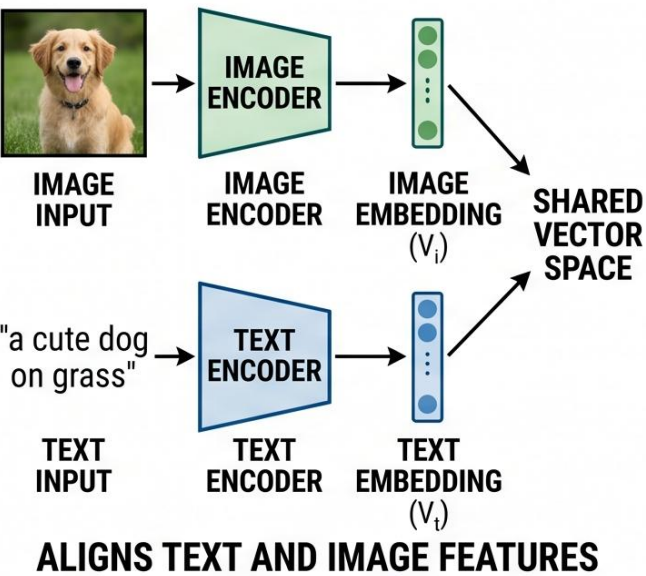


Multimodal Embedding

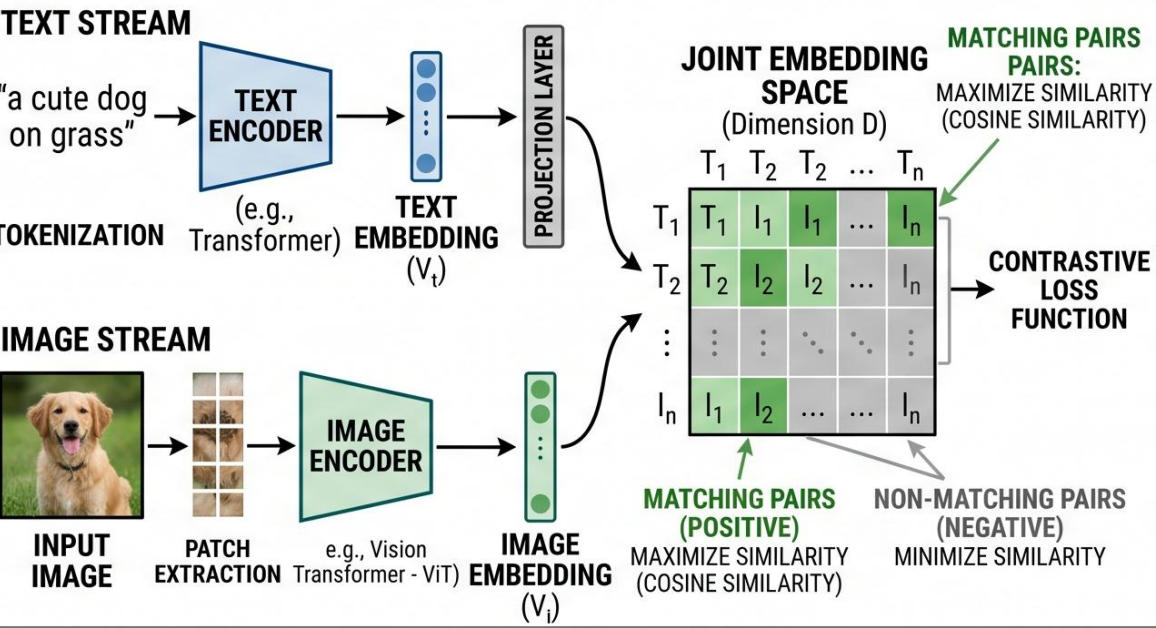
MULTIMODAL EMBEDDING MODEL: TEXT AND IMAGE

LEARNING A SHARED REPRESENTATION SPACE.

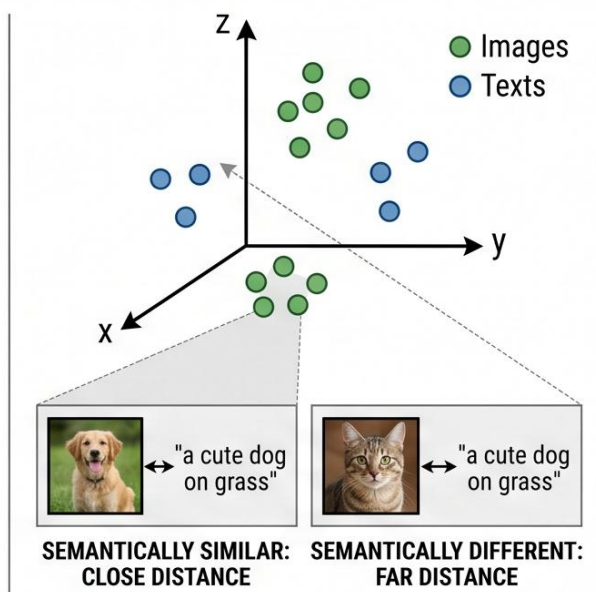
1. OVERVIEW



2. ARCHITECTURE & CONTRASTIVE TRAINING



3. JOINT EMBEDDING SPACE



4. INFERENCE & APPLICATIONS

ZERO-SHOT CLASSIFICATION

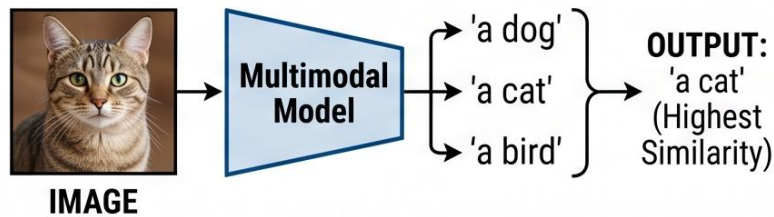


IMAGE-TEXT RETRIEVAL (Semantic Search)

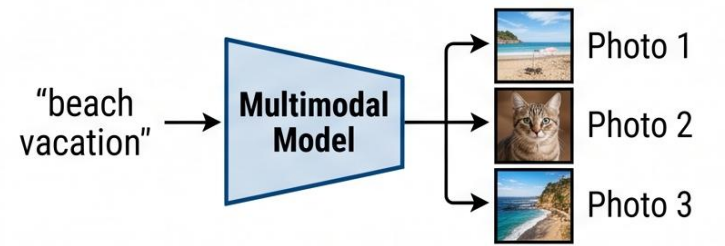


IMAGE CAPTIONING (as a building block)

